

Nav2 原理、模块职责与 ROS2/Unitree SDK 接入教程

面向四足机器人导航系统的总体说明

2026 年 6 月 16 日

目录

第一章 总体概览：Nav2 到底在做什么	4
1.1 Nav2 标准数据流总图	4
1.2 Nav2 的边界	4
第二章 Nav2 和 ROS2 的关系	6
2.1 ROS2 提供通信机制，Nav2 提供导航节点	6
2.2 Nav2 依赖的基础坐标系	6
第三章 Nav2 常用接口：Action、Service 与 Topic	8
3.1 导航任务 Action	8
3.2 路径规划与路径跟踪 Action	8
3.3 恢复行为与辅助行为 Action	9
3.4 常用 Service	10
3.5 核心 Topic	10
第四章 Nav2 核心模块总表	12
第五章 定位、地图与传感器输入模块	13
5.1 Map Server	13
5.1.1 主要 API 与函数角色	13
5.1.2 主要参数	13
5.2 Map Saver	14
5.3 AMCL	14
5.3.1 主要 API 与函数角色	15
5.3.2 主要参数	15
5.4 SLAM	16
5.5 Robot Localization	16
5.5.1 接入 Nav2 时的参数关注点	16
第六章 Costmap：Nav2 怎样理解障碍物	17
6.1 Global Costmap 与 Local Costmap	17

6.2	常见 Layer 插件	17
6.3	Footprint 与 Robot Radius	18
6.4	主要 API、话题与服务	18
6.5	主要参数	18
6.6	Costmap 参数调试时先看什么	19
6.7	障碍物进入 Costmap 的流程图	19
第七章	Planner Server: 全局路径规划	21
7.1	输入与输出	21
7.2	常见 Planner 插件	21
7.3	主要 API 与函数角色	21
7.4	主要参数	22
7.5	Smoother Server: 路径平滑	23
7.6	Planner 不负责什么	23
第八章	Controller Server: 局部路径跟踪与速度生成	24
8.1	输入与输出	24
8.2	常见 Controller 插件	24
8.3	Progress Checker 与 Goal Checker	25
8.4	主要 API 与函数角色	25
8.5	主要参数	26
8.6	Controller 速度生成流程图	27
第九章	BT Navigator: 导航任务的大脑	28
9.1	主要 API 与函数角色	28
9.2	主要参数	29
9.3	NavigateToPose 的典型内部流程	29
第十章	Behavior Server: 恢复动作与辅助动作	31
10.1	主要 API 与参数	31
第十一章	Waypoint Follower 与 Route Server	33
11.1	Waypoint Follower	33
11.2	Route Server	34
11.3	Route Server 的主要 API 与参数	34
第十二章	速度平滑、碰撞监控与生命周期	36
12.1	Velocity Smoother	36
12.2	Collision Monitor	37
12.3	Lifecycle Manager	38

12.4 Lifecycle 主要 API 与参数	38
第十三章 Nav2 模块职责速查	40
第十四章 Nav2 应该怎样接入 ROS2 与 Unitree SDK	41
14.1 总体架构	41
14.2 Unitree Bridge 的上行职责	42
14.3 Unitree Bridge 的下行职责	42
14.4 Nav2 与 Unitree SDK 的完整数据流	43
第十五章 接入检查清单	44
15.1 ROS2 数据是否合格	44
15.2 Nav2 参数是否合格	44
15.3 Unitree Bridge 是否合格	45
第十六章 从总体上记住 Nav2	46

第一章 总体概览：Nav2 到底在做什么

Nav2，全称 Navigation2，是 ROS2 生态里的机器人自主导航框架。它不是一个单独算法，而是一组协同工作的节点、插件、Action、Service、Topic、TF 坐标变换和参数系统。用一句话概括：

Nav2 的输入是“目标点、地图、定位、里程计、传感器障碍物信息”，输出是机器人底盘能够执行的速度命令，通常是 `/cmd_vel`。

因此，理解 Nav2 不要从某一个控制器插件开始，而要先从完整数据流开始。一个标准移动机器人导航系统通常按下面顺序运行：

1. 机器人驱动或 SDK bridge 发布状态：`/odom`、`/tf`、`/scan` 或 `/pointcloud`、`/imu`。
2. 定位模块估计机器人在地图里的位置，例如 AMCL 或 SLAM。
3. costmap 把地图和障碍物转换成可规划的代价地图。
4. planner 根据目标点和全局 costmap 计算全局路径。
5. controller 根据当前位姿、局部 costmap 和全局路径计算短时间速度。
6. smoother、collision monitor 等模块对速度做平滑、限幅或最后安全检查。
7. 机器人驱动或 SDK bridge 把 `/cmd_vel` 转换成底盘 SDK 的运动接口调用。

1.1 Nav2 标准数据流总图

1.2 Nav2 的边界

Nav2 负责“导航决策”，但它通常不直接控制硬件电机，也不直接调用机器人厂商 SDK。Nav2 期望看到的是标准 ROS2 接口：

- 用 TF 表达坐标系关系；
- 用 `/odom` 表达短时间连续运动；
- 用 `/scan` 或 `/pointcloud` 表达障碍物观测；

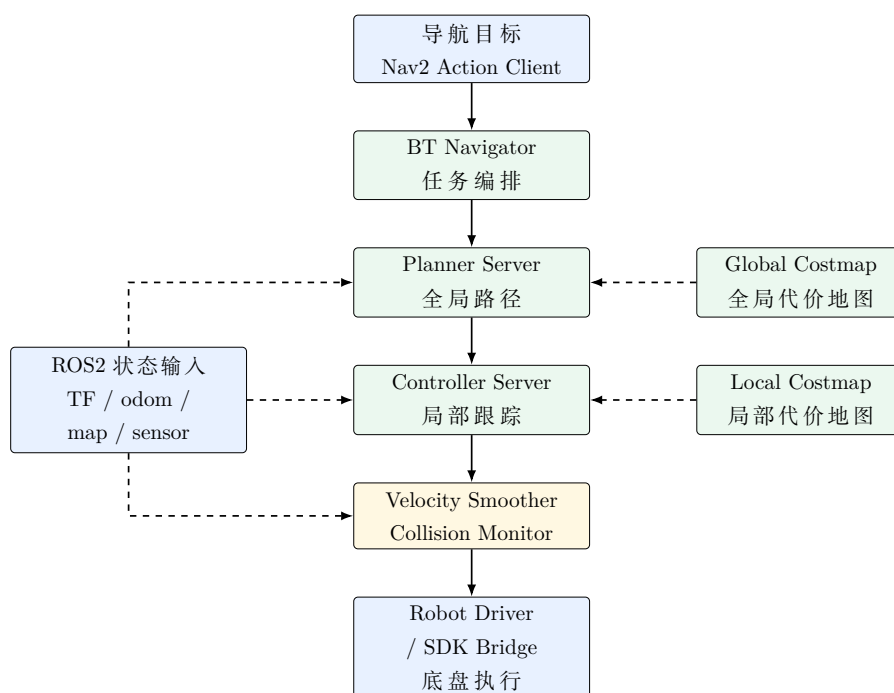


图 1.1: Nav2 的标准闭环：任务层给目标，规划层给路径，控制层给速度，驱动层执行并反馈状态。

- 用 `/cmd_vel` 表达速度命令；
- 用 Action 表达“去某个目标点”“沿某条路径走”等长任务。

机器人厂商 SDK，例如 Unitree SDK，应该放在驱动层或 bridge 层。Bridge 的职责是把 SDK 数据转换成 ROS2 数据，把 ROS2 命令转换成 SDK 调用。这样 Nav2 不需要知道底层机器人来自 Unitree、Clearpath、TurtleBot 还是别的平台。

第二章 Nav2 和 ROS2 的关系

2.1 ROS2 提供通信机制，Nav2 提供导航节点

ROS2 是底层通信和软件组织框架，Nav2 是运行在 ROS2 上的一组导航节点。二者关系可以这样理解：

ROS2 机制	在 Nav2 中的作用
Topic	连续数据流，例如 /odom、/scan、/cmd_vel。
Action	长时间任务，例如导航到目标点、计算路径、跟踪路径。 Action 有 goal、feedback、result，适合导航这种需要持续反馈的任务。
Service	短请求短响应，例如清空 costmap、加载地图、查询路径是否有效。
TF2	维护坐标系树，例如 map -> odom -> base_link。Nav2 的定位、costmap、controller 都依赖 TF。
Parameter	配置插件、频率、速度限制、机器人 footprint、传感器来源等。
Lifecycle Node	管理节点状态：configure、activate、deactivate、cleanup、shutdown。Nav2 大部分核心节点都是 lifecycle 节点。
Pluginlib	让 planner、controller、costmap layer、behavior 等算法可以按参数动态加载。

2.2 Nav2 依赖的基础坐标系

Nav2 中最核心的坐标系一般是三层：

- map：全局地图坐标系。目标点和全局规划一般在这个坐标系下表达。
- odom：连续但可能慢慢漂移的里程计坐标系。短时间控制依赖它的连续性。

- `base_link` 或 `base_footprint`: 机器人本体坐标系。速度命令和传感器外参最终都要落到机器人本体上。

典型 TF 关系是:

```
map -> odom -> base_link -> sensor_frame
```

如果 TF 不连续、时间戳不对、frame 名称不统一，Nav2 的症状通常不是“直接报一个很清楚的错”，而是 `costmap` 不更新、`controller` 不出速度、机器人原地旋转、路径突然跳变。因此 Nav2 调试时要优先检查 TF。

第三章 Nav2 常用接口：Action、Service 与 Topic

用户说的“函数”在 Nav2 里通常不是普通 C++ 函数调用，而是 ROS2 网络接口：Action、Service、Topic。下面按“谁提供、做什么、什么时候调用”来解释。

3.1 导航任务 Action

接口	服务节点	作用
NavigateToPose	bt_navigator	给一个目标位姿，Nav2 自动完成规划、控制、恢复行为和结果返回。这是最常用的高层导航接口。
NavigateThroughPoses	bt_navigator	给多个中间目标，Nav2 按行为树执行穿越多个 Pose 的任务。适合巡检路线、分段导航。
FollowWaypoints	waypoint_follower	给一串 waypoint，让 waypoint follower 按顺序调用导航任务，并可在每个点执行任务插件。它本身不是局部控制器。
FollowGPSWaypoints	waypoint_follower	面向 GPS 坐标 waypoint 的任务接口，常用于户外场景。

3.2 路径规划与路径跟踪 Action

接口	服务节点	作用
ComputePathToPose	planner_server	根据起点、终点和 global costmap 计算一条全局路径。
ComputePathThroughPoses	planner_server	计算穿过多个中间点的全局路径。
FollowPath	controller_server	输入一条路径, controller 根据局部 costmap 和当前 odom/TF 输出速度。
SmoothPath	smoother_server	对规划路径进行平滑处理, 减少尖角或不自然转折。
ComputeRoute	route_server	基于路线图或拓扑图计算路线。适合道路网络、走廊网络、巡检点网络。
ComputeAndTrackRoute	route_server	计算路线并持续跟踪路线执行状态。具体行为随 Nav2 版本和配置变化。

3.3 恢复行为与辅助行为 Action

接口	服务节点	作用
Spin	behavior_server	原地旋转。常用于脱困、重新观察环境或调整朝向。
BackUp	behavior_server	后退一小段距离。常用于局部路径被堵时退出狭窄区域。
DriveOnHeading	behavior_server	沿指定朝向行驶一段距离。
Wait	behavior_server	等待一段时间, 通常作为行为树中的恢复动作。
AssistedTeleop	behavior_server	辅助遥控模式, 可结合碰撞检测做安全限速。

3.4 常用 Service

Service	作用
ClearEntireCostmap	清空整张 costmap。常用于传感器误检、障碍物残影严重时。
ClearCostmapAroundRobot	清空机器人附近区域的 costmap。
ClearCostmapExceptRegion	清空指定区域外的 costmap。
ClearCostmapAroundPose	清空某个位姿附近的 costmap。
GetCostmap	获取当前 costmap 数据，用于调试、可视化或外部分析。
IsPathValid	查询路径当前是否仍然有效。
LoadMap	地图服务器加载地图。
SaveMap	保存地图，常用于 SLAM 建图后落盘。
ManageLifecycleNodes	lifecycle manager 用来启动、停止、复位 Nav2 节点。
SetInitialPose	设置定位初始位姿，常用于 AMCL。

3.5 核心 Topic

Topic	消息类型	作用
/tf、/tf_static	tf2_msgs/TFMessage	坐标系变换。Nav2 几乎所有模块都依赖它。
/odom	nav_msgs/Odometry	机器人短时间连续运动估计。controller 根据它闭环控制。
/scan	sensor_msgs/LaserScan	2D 激光雷达数据，可进入 obstacle layer。
/pointcloud	sensor_msgs/PointCloud2	3D 点云数据，可进入 voxel layer 或 obstacle layer。
/map	nav_msgs/OccupancyGrid	静态地图，通常由 map server 发布。
/cmd_vel	geometry_msgs/Twist	Nav2 输出的底盘速度命令。

Topic	消息类型	作用
/goal_pose	geometry_msgs/PoseStamped	RViz 常用的目标点输入话题。

第四章 Nav2 核心模块总表

下面这张表把 Nav2 常见模块放在同一视角下看：每个模块解决什么问题、外部通常怎样调用它、它依赖或发布哪些话题。后面章节会再展开每个模块的主要 API、插件函数和关键参数。

模块	作用	常见 API、话题与参数入口
map_server	加载静态地图并发布占据栅格。	API: LoadMap; Topic: /map; 参数: yaml_filename、topic_name、frame_id。
map_saver	把当前地图保存为 yaml + 图像文件。	API: SaveMap; 参数: 保存超时、空闲/占据阈值、地图订阅 QoS。
amcl	用地图、激光和里程计估计机器人在 map 中的位置。	API: SetInitialPose; Topic/TF: /scan、/particle_cloud、map -> odom; 参数: frame、粒子数、激光模型。
global_costmap	为全局规划提供地图和障碍物代价。	Topic: /global_costmap/costmap、footprint; Service: 清图、取图; 参数: frame、plugins、footprint、未知区域策略。
local_costmap	为局部控制提供机器人附近动态障碍物代价。	Topic: /local_costmap/costmap、footprint; Service: 清图、取图; 参数: rolling window、尺寸、分辨率、传感器源。
planner_server	根据目标点和 global costmap 计算全局路径。	Action: ComputePathToPose、ComputePathThroughPoses; Service: IsPathValid; 参数: planner 插件、频率、容差、未知区策略。
smoother_server	对全局路径做几何平滑。	Action: SmoothPath; 参数: smoother 插件、迭代次数、收敛阈值、细化次数。
controller_server	把路径、局部环境和当前状态转换为速度命令。	Action: FollowPath; Topic: /cmd_vel、/speed_limit; 参数: controller、progress checker、goal checker、速度阈值。
bt_navigator	用行为树组织规划、控制、恢复和结果返回。	Action: NavigateToPose、NavigateThroughPoses; 参数: BT XML、navigator 插件、BT tick 周期、插件库。
behavior_server	提供旋转、后退、等待等恢复动作。	Action: Spin、BackUp、DriveOnHeading、Wait; 参数: 行为插件、预测时间、速度边界。
waypoint_follower	顺序执行 waypoint，并可在每个点执行任务插件。	Action: FollowWaypoints、FollowGPSWaypoints; 参数: 循环频率、失败策略、waypoint task 插件。
route_server	在路线图/拓扑图上计算路线并支持路线操作。	Action: ComputeRoute、ComputeAndTrackRoute; 参数: 路线图文件、节点到达半径、路线操作、边代价函数。
velocity_smoother	平滑速度，限制速度、加速度和减速度。	Topic: 速度输入/输出，如 /cmd_vel_nav 到 /cmd_vel; 参数: 平滑频率、反馈模式、速度/加速度边界、超时。
collision_monitor	在最终速度输出前做近距离安全过滤。	Topic: 速度输入/输出、/collision_monitor_state; 参数: 安全区、传感器源、输入/输出速度话题。
lifecycle_manager	统一配置、激活、关闭 Nav2 lifecycle 节点。	Service: ManageLifecycleNodes; 参数: 被管理节点列表、自动启动、bond 超时。

第五章 定位、地图与传感器输入模块

5.1 Map Server

`map_server` 的作用是加载并发布二维栅格地图。它通常读取一组地图文件：

- `map.yaml`：描述地图分辨率、原点、阈值、图片路径；
- `map.pgm` 或 `map.png`：实际占据栅格图片。

发布结果通常是 `/map`。`global costmap` 的 `static layer` 会读取这张地图，把墙、不可通行区域和未知区域转换成代价值。

5.1.1 主要 API 与函数角色

接口 / 函数角色	说明
<code>LoadMap</code>	运行时重新加载地图。典型用途是从一张室内地图切换到另一张地图，或者建图后加载保存好的地图。
<code>/map</code>	发布 <code>nav_msgs/OccupancyGrid</code> ，供 <code>static layer</code> 、 <code>AMCL</code> 、 <code>RViz</code> 使用。
<code>SaveMap</code>	由 <code>map_saver</code> 提供，把当前 <code>/map</code> 保存成图片和 <code>yaml</code> 。
<code>loadMapYaml / loadMapFromFile</code>	<code>map server</code> 内部读取 <code>yaml</code> 和图像文件的核心功能。教程使用时不直接调用 C++ 函数，而是通过参数或 <code>service</code> 间接使用。

5.1.2 主要参数

参数	作用
yaml_filename	地图 yaml 文件路径。为空时节点可以启动但不加载初始地图，后续可用 LoadMap 加载。
topic_name	地图发布话题，默认常见为 /map。如果改名，AMCL 和 static layer 也要同步改。
frame_id	地图消息的 frame，通常是 map。它必须和 Nav2 的 global_frame 保持一致。
map_subscribe_transient_local	订阅地图时是否使用 transient local QoS。static layer 常设为 true，使 late subscriber 也能收到已经发布的地图。

5.2 Map Saver

map_saver 和 map_server 是一对互补模块。map server 负责“读地图并发布”，map saver 负责“把当前地图保存到文件”。SLAM 建图完成后，常用 map saver 把 /map 保存成 yaml 和 pgm/png，之后再由 map server 加载。

接口 / 参数	说明
SaveMap	保存地图的 service。请求里通常包含地图话题、保存文件名、图像格式、free/occupied 阈值。
map_subscribe_transient_local	保存地图时订阅 /map 的 QoS。若地图只发布一次，通常需要 transient local。
save_map_timeout	等待地图数据和写文件的超时时间。地图较大或磁盘较慢时需要留足。
free_thresh_default、occupied_thresh_default	默认空闲/占据阈值。它们决定 OccupancyGrid 中的概率值怎样落到地图图片里。

5.3 AMCL

amcl 是基于粒子滤波的二维定位模块。它的输入通常包括：

- 静态地图 /map；
- 激光雷达 /scan；
- 里程计 TF 或 /odom；

- 初始位姿。

它输出的是 map -> odom 这段 TF。简单理解：odom 坐标系靠轮速、IMU、腿式里程计等短时间积分保持连续；map 坐标系靠地图匹配纠正长期漂移。

5.3.1 主要 API 与函数角色

接口 / 函数角色	说明
SetInitialPose	设置初始位姿。AMCL 是粒子滤波，初始位姿给得越合理，收敛越快。
/amcl_pose	输出估计位姿，常用于调试或外部记录。
/particle_cloud	输出粒子云，可在 RViz 中观察定位是否收敛。粒子云很散说明定位不确定。
map -> odom TF	AMCL 最重要的输出。它不直接替代 odom，而是修正 odom 在 map 中的位置。
运动模型更新	根据里程计运动更新粒子预测。参数 alpha1 到 alpha5 控制运动噪声。
激光观测更新	用激光与地图匹配给粒子打分。参数 laser_model_type、z_hit、z_rand 等影响匹配模型。

5.3.2 主要参数

参数	作用
base_frame_id	机器人本体 frame，常见为 base_link 或 base_footprint。
odom_frame_id	里程计 frame，通常为 odom。AMCL 使用 odom 预测短时间运动。
global_frame_id	全局 frame，通常为 map。AMCL 输出 map -> odom。
scan_topic	激光雷达话题。AMCL 主要基于 2D laser scan 工作。
min_particles、 max_particles	粒子数量范围。数量越大越稳但计算越重。
update_min_d、 update_min_a	位移或角度变化超过阈值才触发滤波更新。阈值过大定位更新慢，过小计算负担增加。

参数	作用
<code>laser_model_type</code>	激光模型，常见为 <code>likelihood_field</code> 。影响激光与地图的匹配方式。
<code>transform_tolerance</code>	TF 容忍时间。传感器和 TF 延迟较大时需要合理增大，但过大也会掩盖时间同步问题。

5.4 SLAM

SLAM 表示 simultaneous localization and mapping，即同时定位与建图。SLAM 模块不是 Nav2 核心包的一部分，但经常和 Nav2 一起用。它的任务是机器人边走边估计自身轨迹，同时生成地图。

在 Nav2 系统中，SLAM 的常见输出是：

- `/map`：实时生成的地图；
- `map -> odom`：定位修正；
- 轨迹或位姿估计。

如果已经有离线地图，则常用 `map server + AMCL`；如果没有地图，则常用 SLAM 建图，之后再保存地图用于导航。

5.5 Robot Localization

`robot_localization` 不是 Nav2 核心包，但常作为 Nav2 前端状态估计使用。它可以融合轮速、IMU、视觉里程计、GPS、腿式里程计等数据，输出更稳定的 `/odom` 和 TF。

对四足机器人来说，SDK 内部给出的机体位姿、速度和 IMU 数据不一定能直接满足 Nav2 的 frame、时间戳和协方差要求，所以常常需要一个状态估计或 bridge 层做标准化。

5.5.1 接入 Nav2 时的参数关注点

- SLAM 或定位模块必须提供 `map -> odom`，机器人驱动或状态估计必须提供 `odom -> base_link`。
- `robot_localization` 常用输入是 `/imu`、轮速或 SDK `odom`，常用输出是 `/odometry/filtered` 和 `odom -> base_link`。
- 状态估计输出的协方差不能完全乱填。协方差过小会让后端过度相信错误数据，协方差过大会让定位不稳定。

第六章 Costmap: Nav2 怎样理解障碍物

Costmap 是 Nav2 的核心数据结构。它把地图、传感器障碍物、机器人尺寸和安全距离统一成一张二维代价地图。代价值越高，表示越不适合通行。

6.1 Global Costmap 与 Local Costmap

Costmap	作用
Global Costmap	面向全局路径规划，覆盖范围通常较大，主要使用静态地图和较慢更新的障碍物信息。planner server 依赖它。
Local Costmap	面向局部避障和轨迹跟踪，覆盖机器人附近一小块区域，更新频率通常更高。controller server 依赖它。

6.2 常见 Layer 插件

Layer	作用
StaticLayer	读取 /map，把静态地图中的墙体、占据区域加入 costmap。
ObstacleLayer	读取 /scan 或 /pointcloud，把传感器看到的障碍物标记到 costmap，也可以清除已经看不到的区域。
VoxelLayer	用三维体素网格处理 3D 点云，再投影到二维 costmap，适合 RGB-D、3D LiDAR 或深度点云。
InflationLayer	对障碍物做膨胀。即使墙体只占一格，也会在周围形成逐渐降低的代价，给机器人留安全距离。

Layer	作用
CostmapFilter	根据过滤地图实现 keepout zone、速度限制区等高级约束。

6.3 Footprint 与 Robot Radius

Nav2 必须知道机器人占地尺寸。常见配置方式有两种：

- `robot_radius`：用圆近似机器人，参数少，适合圆形或近似圆形底盘。
- `footprint`：用多边形描述机器人外轮廓，适合矩形底盘、四足机器人、带突出传感器的平台。

Footprint 影响路径是否可通行，也影响局部控制器是否认为某条轨迹会碰撞。`footprint` 设置过大，机器人会觉得很多地方过不去；设置过小，又可能贴障太近。

6.4 主要 API、话题与服务

接口	说明
<code>/global_costmap/costmap</code> 、 <code>/local_costmap/costmap</code>	发布 costmap，用于 RViz 或外部调试。不同 launch 可能带命名空间。
<code>/published_footprint</code>	发布机器人 footprint 在地图中的位置，便于确认 footprint 是否正确。
<code>ClearEntireCostmap</code>	清空整张 costmap。障碍物残影严重时常用。
<code>ClearCostmapAroundRobot</code>	清空机器人附近区域，常用于局部误检恢复。
<code>GetCostmap</code>	获取当前 costmap 栅格数据，适合调试或算法分析。
Layer 插件接口	各 layer 通过 pluginlib 加载，核心角色是 <code>updateBounds</code> 和 <code>updateCosts</code> ：前者决定需要更新哪片区域，后者写入代价值。

6.5 主要参数

参数	作用
<code>global_frame</code>	costmap 工作坐标系。global costmap 常用 <code>map</code> ，local costmap 常用 <code>odom</code> 。
<code>robot_base_frame</code>	机器人本体 frame，必须能通过 TF 转到 costmap frame。
<code>update_frequency</code> 、 <code>publish_frequency</code>	更新频率和发布频率。local costmap 通常比 global costmap 更高。
<code>rolling_window</code>	local costmap 常设为 <code>true</code> ，使窗口跟随机器人移动。
<code>width</code> 、 <code>height</code> 、 <code>resolution</code>	costmap 尺寸和分辨率。窗口越大感知范围越广，但计算越重。
<code>plugins</code>	layer 插件列表，例如 <code>static_layer</code> 、 <code>obstacle_layer</code> 、 <code>voxel_layer</code> 、 <code>inflation_layer</code> 。
<code>observation_sources</code>	传感器来源列表。每个 source 下还要配置 <code>topic</code> 、 <code>data_type</code> 、 <code>marking</code> 、 <code>clearing</code> 。
<code>obstacle_max_range</code> 、 <code>raytrace_max_range</code>	前者决定多远的点会标记障碍物，后者决定多远范围内能清除自由空间。
<code>inflation_radius</code> 、 <code>cost_scaling_factor</code>	控制障碍物膨胀半径和代价衰减。它们直接影响机器人离障碍物多远。
<code>always_send_full_costmap</code>	是否每次发布完整 costmap。调试方便但带宽更高。

6.6 Costmap 参数调试时先看什么

Costmap 参数很多，但排查问题时可以按下面顺序看：

1. TF 是否连通：sensor frame 能否转到 `global_frame` 和 `robot_base_frame`。
2. 传感器源是否进入 layer：`observation_sources`、`topic`、`data_type` 是否一致。
3. 机器人尺寸是否合理：`robot_radius` 或 `footprint` 是否比真实外形略保守。
4. 障碍物范围是否合理：`obstacle_max_range` 太小会漏障，太大可能引入远处噪声。
5. 膨胀是否合理：`inflation_radius` 太大导致“到处过不去”，太小导致贴障太近。

6.7 障碍物进入 Costmap 的流程图

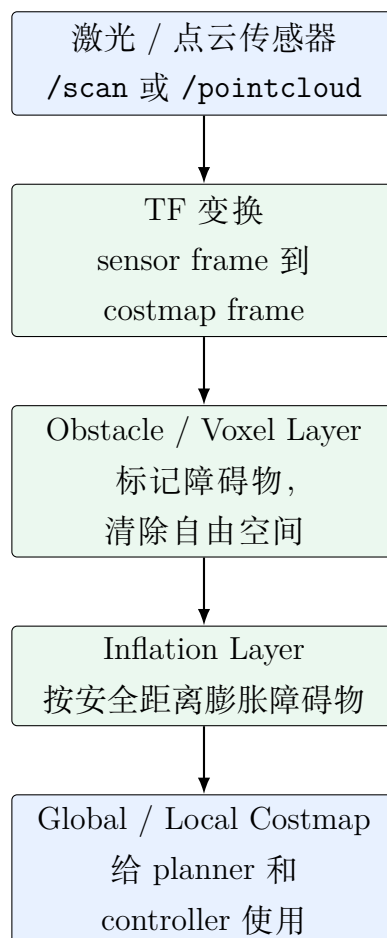


图 6.1: 传感器障碍物转换为 costmap 的基本过程。

第七章 Planner Server: 全局路径规划

`planner_server` 负责“从当前位置到目标点走哪条路”。它不直接输出速度，而是输出一条路径，类型通常是 `nav_msgs/Path`。

7.1 输入与输出

项目	内容
输入	起点、目标点、global costmap、TF、planner 插件参数。
输出	一条全局路径，通常是一串 PoseStamped。
典型 Action	ComputePathToPose、ComputePathThroughPoses。
调用者	BT Navigator、Route Server、外部程序或测试脚本。

7.2 常见 Planner 插件

插件	特点
NavFn	经典二维栅格全局规划器，基于 Dijkstra/A* 思路，适合差速或全向底盘的普通室内导航。
SmacPlanner2D	面向二维栅格路径，性能和路径质量更现代。
SmacPlannerHybrid	考虑车辆运动学约束，适合 Ackermann、汽车式底盘或需要非完整约束的机器人。
ThetaStar	倾向生成更接近任意角度直线的路径，减少栅格锯齿。

7.3 主要 API 与函数角色

接口 / 函数角色	说明
<code>ComputePathToPose</code>	给定一个目标位姿，返回从当前位姿或指定起点到目标的 <code>nav_msgs/Path</code> 。这是最常见的全局规划 Action。
<code>ComputePathThroughPoses</code>	给定多个中间目标点，计算经过这些点的路径。适合“必须经过某些站点”的任务。
<code>IsPathValid</code>	检查一条路径在当前 <code>costmap</code> 中是否仍然有效。行为树可用它决定是否需要重新规划。
<code>createPlan</code>	planner 插件的核心函数角色：输入起点、终点和 <code>costmap</code> ，输出一条路径。不同插件实现不同搜索策略。
<code>configure</code> 、 <code>activate</code> 、 <code>deactivate</code> 、 <code>cleanup</code>	lifecycle/plugin 生命周期函数角色。加载插件、获取 <code>costmap</code> 、激活 action server、释放资源都发生在这些阶段。

7.4 主要参数

参数	作用
<code>planner_plugins</code>	planner 插件 ID 列表。例如 <code>GridBased</code> 。Action goal 中的 <code>planner_id</code> 会匹配这里的 ID。
<code>expected_planner_frequency</code>	期望规划频率，用于诊断规划器是否过慢。它不是强制控制周期，但会影响日志和性能判断。
<code>costmap_update_timeout</code>	等待 global <code>costmap</code> 更新的最长时间。传感器或 TF 慢时，规划可能卡在这里。
<code>allow_partial_planning</code>	多目标规划时是否允许部分路径成功。对 <code>through poses</code> 场景有意义。
<code>GridBased.plugin</code>	指定某个 planner ID 对应的插件类型，例如 <code>nav2_navfn_planner::NavfnPlanner</code> 。
<code>tolerance</code>	目标点附近可接受的搜索容差。目标落在障碍物或不可达格子时，规划器可尝试附近可达点。
<code>use_astar</code>	NavFn 等插件的选项：是否使用 A* 搜索而不是默认 Dijkstra 风格搜索。

参数	作用
allow_unknown	是否允许路径穿过未知区域。地图不完整时打开更激进，关闭更保守。

7.5 Smoother Server：路径平滑

smoother_server 位于 planner 和 controller 之间。planner 输出的路径可能带有栅格折线、锯齿或局部不连续，smoother server 会对路径做几何平滑，让 controller 更容易跟踪。

接口 / 参数	说明
SmoothPath	输入一条路径，输出平滑后的路径。行为树中可在 ComputePath 后调用。
smoother_plugins	路径平滑插件 ID 列表，例如 simple_smoother。
tolerance	平滑迭代停止阈值。越小越追求收敛，但计算可能增加。
max_its	最大迭代次数，防止平滑过程无限运行。
refinement_num	细化次数。次数越多路径更平滑，但耗时更高。
enforce_path_inversion	是否保留路径中的前进/后退方向切换，适合有倒车路径语义的机器人。

7.6 Planner 不负责什么

Planner 只回答“从全局看，应该走哪里”。它不负责每一时刻的速度，不直接做近距离急停，也不直接调用底盘 SDK。真正把路径转成速度的是 controller server。

第八章 Controller Server：局部路径跟踪与速度生成

`controller_server` 负责“现在应该怎么动”。它读取当前 odom/TF、local costmap 和全局路径，输出 `/cmd_vel`。

8.1 输入与输出

项目	内容
输入	当前位姿、当前速度、全局路径、local costmap、机器人 footprint、controller 参数。
输出	<code>geometry_msgs/Twist</code> ，即线速度和角速度。
典型 Action	<code>FollowPath</code> 。
调用者	BT Navigator 或外部程序。

8.2 常见 Controller 插件

插件	作用与特点
DWB	Dynamic Window Based controller。采样多组候选速度，预测短时轨迹，用 critic 打分选择最优速度。参数较多，但可解释性强。
RegulatedPurePursuit	纯追踪控制器的增强版本，参数相对少，适合路径跟踪清晰、底盘响应稳定的场景。

插件	作用与特点
MPPI	Model Predictive Path Integral controller。基于采样优化的局部控制器，适合较复杂动态约束，但计算和调参要求更高。

8.3 Progress Checker 与 Goal Checker

controller_server 里除了 controller 插件，还常配置两个检查器：

- Progress Checker：判断机器人是否在持续接近目标。如果长时间没有有效移动，可能触发失败或恢复行为。
- Goal Checker：判断机器人是否已经到达目标。它会检查位置误差和角度误差是否小于阈值。

8.4 主要 API 与函数角色

接口 / 函数角色	说明
FollowPath	controller server 的核心 Action。输入 path、controller_id、goal_checker_id、path_handler_id，执行期间持续输出速度。
/cmd_vel	controller 的典型速度输出。实际系统中常重映射为 /cmd_vel_nav，再进入 velocity smoother 或 collision monitor。
/speed_limit	可选速度限制输入。速度过滤区或外部安全模块可通过它动态限制最大速度。
computeVelocityCommands	controller 插件核心函数角色：根据当前位姿、速度、路径和 local costmap 计算下一帧速度。
check / reset	Progress Checker 的函数角色：判断是否持续前进，并在新目标开始时重置状态。
isGoalReached	Goal Checker 的函数角色：判断位置误差、角度误差是否满足到达条件。
transformGlobalPlan / path handler	将全局路径裁剪并变换到 controller 使用的局部 frame，避免 controller 每次处理整条长路径。

8.5 主要参数

参数	作用
<code>controller_frequency</code>	控制循环频率。频率越高响应越快，但要求 odom、TF、costmap 和底盘执行都跟得上。
<code>controller_plugins</code>	controller 插件 ID 列表，例如 FollowPath。Action goal 中的 <code>controller_id</code> 会匹配这里。
<code>progress_checker_plugins</code>	进度检查器插件列表。常见 SimpleProgressChecker 会检查一定时间内是否移动超过半径阈值。
<code>goal_checker_plugins</code>	到达检查器插件列表。常见 SimpleGoalChecker 配置 xy/yaw 容差。
<code>path_handler_plugins</code>	路径处理插件列表，用于局部裁剪、可行性检查或路径段选择。
<code>failure_tolerance</code>	controller 短时失败容忍时间。容忍时间内可继续尝试，超过后返回失败给行为树。
<code>costmap_update_timeout</code>	等待 local costmap 更新的最长时间。local costmap 不更新时，速度生成应被阻止或降级。
<code>min_x_velocity_threshold</code> 、 <code>min_y_velocity_threshold</code> 、 <code>min_theta_velocity_threshold</code>	小速度阈值。低于阈值的速度可视为 0，减少微小噪声导致的抖动。
<code>speed_limit_topic</code>	接收动态速度限制的话题名。

如果使用 MPPI，还会看到一组更像优化器的参数：

MPPI 参数	作用
<code>time_steps</code> 、 <code>model_dt</code>	预测多少步、每步多长时间，二者共同决定预测时域。
<code>batch_size</code>	每轮采样的候选控制序列数量。越大越稳，但计算量越高。
<code>vx_max</code> 、 <code>vx_min</code> 、 <code>vy_max</code> 、 <code>wz_max</code>	线速度和角速度边界。必须符合机器人真实能力。

MPPI 参数	作用
ax_max、ax_min、ay_max、az_max	加速度边界。设置过大动作激进，过小会跟踪迟钝。
critics	代价函数插件列表，用来评价候选轨迹，例如避障、贴近路径、朝向目标、速度偏好等。
motion_model	运动模型，例如 differential、omni、ackermann。 四足机器人如果按平面速度控制，通常更接近 omni 或可侧向模型。

8.6 Controller 速度生成流程图

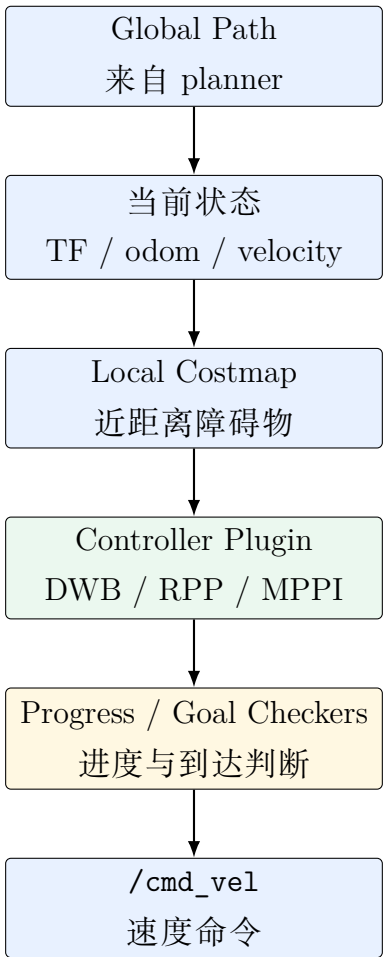


图 8.1: Controller server 把路径和局部环境转换为速度命令。

第九章 BT Navigator: 导航任务的大脑

`bt_navigator` 是 Nav2 的任务编排层。BT 是 Behavior Tree，即行为树。行为树把一个复杂导航任务拆成多个可组合节点，例如：

- 计算路径；
- 跟踪路径；
- 判断目标是否更新；
- 判断路径是否失效；
- 清空 costmap；
- 原地旋转或后退恢复；
- 失败时重试或退出。

行为树的好处是：导航任务不再写死在一个大函数里，而是由 XML 配置描述。不同机器人可以复用同一个 planner/controller，但用不同的行为树策略。

9.1 主要 API 与函数角色

接口 / 函数角色	说明
<code>NavigateToPose</code>	单目标导航 Action。输入目标 pose，BT Navigator 在内部调用 planner、controller、recovery，最终返回成功或失败。
<code>NavigateThroughPoses</code>	多目标导航 Action。输入一组 poses，行为树按策略依次或整体处理。
BT XML	行为树文件。里面的节点会映射到具体 Action、Condition、Control、Decorator 插件。
<code>ComputePathToPose</code> BT 节点	行为树里的规划动作节点，实际调用 planner server。

接口 / 函数角色	说明
FollowPath BT 节点	行为树里的路径跟踪动作节点，实际调用 controller server。
RecoveryNode / RoundRobin	行为树控制节点，用来组织失败重试、清图、旋转、后退等恢复策略。
GoalUpdated、IsPathValid 等条件节点	判断目标是否变化、路径是否仍有效、是否需要重新规划。

9.2 主要参数

参数	作用
global_frame、robot_base_frame	BT Navigator 判断位姿和反馈时使用的 frame。必须与 TF 和 costmap 配置一致。
odom_topic	用于导航反馈和部分行为树节点的里程计来源。
bt_loop_duration	行为树 tick 周期，单位通常是毫秒。太慢会让任务响应迟钝，太快会增加 CPU 压力。
default_server_timeout	等待下游 Action/Service 的默认超时时间。planner/controller/behavior 卡住时会影响失败判定。
navigators	启用哪些 navigator 插件，例如 navigate_to_pose、navigate_through_poses。
default_nav_to_pose_bt_xml、default_nav_through_poses_bt_xml	默认行为树 XML 文件路径。改这两个参数就能替换导航策略。
plugin_lib_names	BT 节点插件库列表。XML 中用到的节点必须能在这里加载到。
bt_search_directories	搜索行为树 XML 或插件相关资源的目录。

9.3 NavigateToPose 的典型内部流程

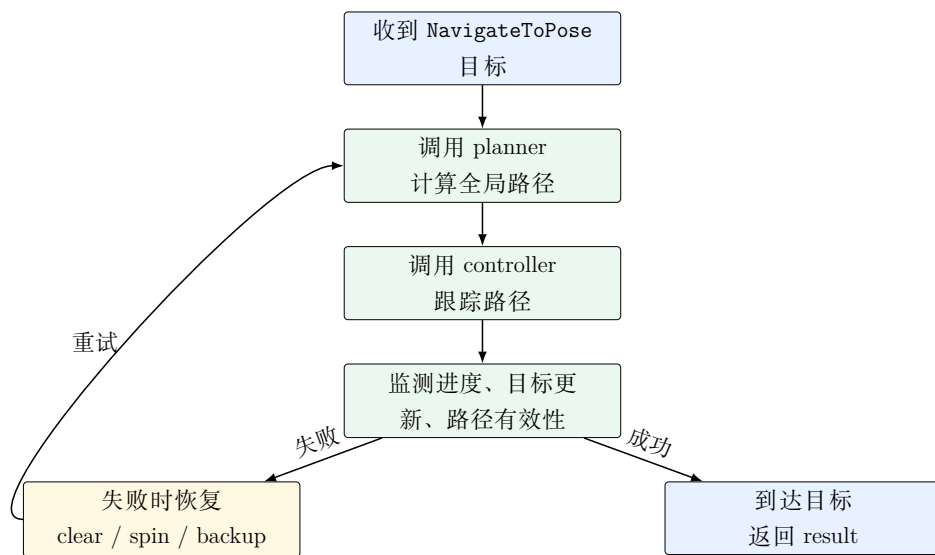


图 9.1: BT Navigator 不是一个单一控制算法，而是导航任务编排器。

第十章 Behavior Server: 恢复动作与辅助动作

`behavior_server` 提供一些短动作，通常由 BT Navigator 在导航异常时调用。它们不是全局规划，也不是主控制器，而是“脱困和辅助行为”。

行为	使用场景
Spin	机器人需要重新扫描周围环境，或调整朝向以继续规划。
BackUp	前方被局部障碍物堵住时，后退给 controller 留出空间。
DriveOnHeading	沿指定方向短距离移动，常用于简单恢复或行为树策略。
Wait	等待动态障碍物离开，或给传感器/定位恢复时间。
AssistedTeleop	人工遥控时加入安全约束。

10.1 主要 API 与参数

接口 / 参数	说明
Spin	原地旋转指定角度。常用于重新观察环境或从局部死锁中恢复。
BackUp	后退指定距离。前方局部空间不足时可用。
DriveOnHeading	沿指定 heading 行驶一段距离，适合简单脱困动作。
Wait	等待指定时间，不产生运动。动态障碍物可能自行离开时很有用。
AssistedTeleop	遥控辅助动作，在人工控制时仍保留一定安全约束。
behavior_plugins	启用的行为插件列表。BT XML 中调用某个行为前，behavior server 必须加载对应插件。

接口 / 参数	说明
<code>cycle_frequency</code>	行为执行循环频率。影响恢复动作速度更新和碰撞检查频率。
<code>local_costmap_topic</code> 、 <code>global_costmap_topic</code>	行为执行时用于碰撞预测或安全输入的 <code>costmap</code> 输入。
<code>local_footprint_topic</code> 、 <code>global_footprint_topic</code>	<code>footprint</code> 输入，用于判断恢复动作是否可能碰撞。
<code>simulate_ahead_time</code>	恢复动作向前预测的时间范围。越大越保守，但可能过早判定不可执行。
<code>max_rotational_vel</code> 、 <code>min_rotational_vel</code> 、 <code>rotational_acc_lim</code>	<code>Spin</code> 等旋转行为的速度和加速度边界。

第十一章 Waypoint Follower 与 Route Server

11.1 Waypoint Follower

`waypoint_follower` 的职责是执行一串 `waypoint`。它通常不直接生成底盘速度，而是对每个 `waypoint` 调用 `NavigateToPose` 或相关导航动作。它可以在到达某个 `waypoint` 后执行任务插件，例如拍照、等待、记录数据等。

因此，Waypoint Follower 是“任务层”的模块，不是“局部避障控制器”。如果使用 Nav2 官方 `waypoint follower`，底层仍然通常需要 `planner`、`controller`、`costmap`、`BT navigator` 等模块共同工作。

接口 / 参数	说明
<code>FollowWaypoints</code>	输入一组 <code>PoseStamped</code> ，逐个导航到 <code>waypoint</code> 。失败点会记录在结果中。
<code>FollowGPSWaypoints</code>	面向 GPS <code>waypoint</code> 的接口，通常需把经纬度转换到导航坐标系。
<code>NavigateToPose client</code>	<code>waypoint follower</code> 内部通常作为 <code>NavigateToPose</code> 的客户端，把每个 <code>waypoint</code> 交给 <code>BT Navigator</code> 。
<code>loop_rate</code>	<code>waypoint</code> 执行循环频率。它不是底层控制频率，而是任务层检查频率。
<code>stop_on_failure</code>	某个 <code>waypoint</code> 失败时是否停止整个任务。 <code>false</code> 时可继续尝试后续点。
<code>waypoint_task_executor_plugin</code>	到达 <code>waypoint</code> 后执行的任务插件，例如等待、拍照、读取传感器。
<code>waypoint_pause_duration</code>	<code>wait</code> 插件常用参数，到达点后等待多久。

11.2 Route Server

route_server 面向“路线图”或“拓扑图”的导航。它适合道路网络、走廊网络、固定巡检路线、站点图等场景。它解决的问题不是“每个栅格怎么走”，而是“从哪个节点到哪个节点，沿哪条边走”。

可以把它理解为比普通 waypoint 更结构化的路线层：

- waypoint list 是一串点；
- route graph 是节点和边组成的网络；
- route server 可以在网络上查询路线，并把路线交给后续 planner/controller 执行。

11.3 Route Server 的主要 API 与参数

接口 / 参数	说明
ComputeRoute	在拓扑图上计算从起点到终点的路线，可输出 route，也可输出转换后的 path。
ComputeAndTrackRoute	计算路线并跟踪路线。适合拓扑路线执行和运行中重路由。
graph_filepath	路线图文件路径。没有路线图时，route server 无法发挥拓扑规划能力。
radius_to_achieve_node	判断已经到达 route node 的半径。太小会导致节点难以判定到达。
boundary_radius_to_achieve_node	靠近边界或特殊区域时的节点到达半径。
smooth_corners	是否平滑路线中的拐角，减少节点连接处的生硬转向。
operations	route operations 插件列表，例如重路由、速度限制、碰撞监控等路线级操作。
edge_cost_functions	边代价函数列表，例如距离代价、costmap 代价。它决定拓扑图上“哪条边更好”。

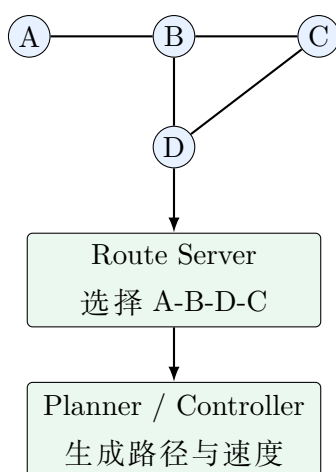


图 11.1: Route server 处理拓扑路线，后续仍可交给规划和控制模块执行。

第十二章 速度平滑、碰撞监控与生命周期

12.1 Velocity Smoother

`velocity_smoother` 位于 `controller` 输出之后、机器人驱动之前。它的作用是把突变速度变得更平滑，同时限制最大速度、最大加速度、最大减速度。

典型数据流是：

```
controller_server -> /cmd_vel_nav -> velocity_smoother -> /cmd_vel
```

在真实机器人上，它可以减少底盘抖动，避免控制器某一帧输出过大速度导致机器人动作不自然。

接口 / 参数	说明
输入速度话题	通常来自 <code>controller server</code> ，例如 <code>/cmd_vel_nav</code> 。实际名称由 <code>remap</code> 或 <code>cmd_vel_in</code> 类参数决定。
输出速度话题	发给下一层安全过滤或底盘 <code>bridge</code> ，例如 <code>/cmd_vel_smoothed</code> 或 <code>/cmd_vel</code> 。
<code>smoothing_frequency</code>	平滑器运行频率。通常应高于或接近 <code>controller</code> 输出频率。
<code>feedback</code>	速度反馈模式。 <code>OPEN_LOOP</code> 只根据上一条命令估计， <code>CLOSED_LOOP</code> 会参考 <code>odom</code> 。
<code>max_velocity</code> 、 <code>min_velocity</code>	<code>x</code> 、 <code>y</code> 、 <code>theta</code> 三个方向的速度上下界。
<code>max_accel</code> 、 <code>max_decel</code>	<code>x</code> 、 <code>y</code> 、 <code>theta</code> 三个方向的加速度和减速度边界。
<code>odom_topic</code> 、 <code>odom_duration</code>	<code>CLOSED_LOOP</code> 模式下使用的 <code>odom</code> 话题和速度估计时间窗口。
<code>deadband_velocity</code>	死区速度。小于死区的速度被压为 0，可减少底盘轻微抖动。

接口 / 参数	说明
<code>velocity_timeout</code>	超过该时间未收到新速度命令时输出停止。真实机器人必须重视这个参数。
<code>scale_velocities</code>	当某一方向超限时，是否按比例缩放其它方向速度，保持速度向量形状。

12.2 Collision Monitor

`collision_monitor` 是最后一道安全过滤层。它读取机器人周围传感器数据，根据预设安全区判断是否减速、停止或限制某些方向的速度。

它和 `costmap` 的区别是：

- `costmap` 服务于规划和控制；
- `collision monitor` 服务于最终速度安全过滤；
- `costmap` 可以帮助机器人绕障；
- `collision monitor` 更像紧急刹车或安全限速器。

接口 / 参数	说明
<code>/collision_monitor_state</code>	发布碰撞监控状态，便于观察当前是否被限速、停止或禁用。
<code>cmd_vel_in_topic</code> 、 <code>cmd_vel_out_topic</code>	输入/输出速度话题。 <code>collision monitor</code> 通常放在最终发给底盘之前。
<code>base_frame_id</code> 、 <code>odom_frame_id</code>	碰撞预测所需 frame。TF 不稳定时，安全过滤会变得不可靠。
<code>polygons</code>	安全区域列表。每个 polygon 可以配置 stop、slowdown、limit、approach 等动作。
<code>observation_sources</code>	传感器来源，如 scan、pointcloud、range。至少要有能覆盖近距离障碍物的数据。
<code>time_before_collision</code>	approach 类型常用参数：按当前速度预测多久以内会碰撞。
<code>simulation_time_step</code>	预测碰撞时的仿真步长。越小越细，但计算量更高。

接口 / 参数	说明
min_points	区域内至少出现多少障碍点才触发动作。设置过低容易被噪声触发，过高可能漏检细小障碍物。
source_timeout、 stop_pub_timeout	传感器超时和停止命令发布超时。真实机器人上应保守配置。

12.3 Lifecycle Manager

Nav2 核心节点大多是 lifecycle node。它们不是一启动就能用，而是经历下面状态：

- 1. unconfigured：节点存在，但未加载参数和资源；
- 2. inactive：已经配置，但还没有开始发布、订阅或执行任务；
- 3. active：正式工作；
- 4. finalized：关闭。

lifecycle_manager 负责按顺序配置和激活这些节点。常见被管理节点包括：

```
controller_server
smoother_server
planner_server
route_server
behavior_server
bt_navigator
waypoint_follower
velocity_smoother
collision_monitor
```

12.4 Lifecycle 主要 API 与参数

接口 / 参数	说明
ManageLifecycleNodes	lifecycle manager 对外提供的管理 service，可执行 startup、shutdown、pause、resume、reset 等命令。
节点 lifecycle services	每个 lifecycle node 自己也有 configure、activate、deactivate、cleanup、shutdown 等状态转换服务。

接口 / 参数	说明
<code>node_names</code>	被管理节点列表。顺序很重要，通常先 <code>costmap/planner/controller</code> ，再 <code>navigator</code> 和上层任务。
<code>autostart</code>	是否启动后自动配置并激活所有节点。实验调试时也可关闭，手动逐个启动。
<code>bond_timeout</code>	<code>lifecycle manager</code> 和被管理节点之间 <code>bond</code> 断开的超时时间。节点挂掉后可快速发现。
<code>bond_heartbeat_period</code>	<code>bond</code> 心跳周期。周期越短发现故障越快，但通信更频繁。
<code>attempt_respawn_</code> <code>reconnection、</code> <code>bond_respawn_max_duration</code>	节点支持 <code>respawn</code> 时， <code>manager</code> 是否尝试重新连接以及最大等待时间。

第十三章 Nav2 模块职责速查

模块	类型	核心职责
map_server	地图	加载并发布静态栅格地图。
amcl	定位	根据地图、激光和里程计估计 map -> odom。
planner_server	规划	计算从当前点到目标点的全局路径。
controller_server	控制	沿路径生成实时速度命令。
smoother_server	路径处理	平滑 planner 输出的路径。
bt_navigator	任务编排	用行为树组织规划、控制、恢复和结果返回。
behavior_server	恢复行为	提供 spin、backup、wait 等动作。
waypoint_follower	多点任务	按顺序执行 waypoint，并可运行 waypoint task 插件。
route_server	拓扑路线	在路线图上计算路线，适合固定道路或巡检网络。
global_costmap	全局环境	给 planner 提供地图和全局障碍物代价。
local_costmap	局部环境	给 controller 提供机器人附近障碍物代价。
velocity_smoother	速度处理	平滑速度、限制加速度和减速度。
collision_monitor	安全过滤	根据近距离传感器对最终速度做停止或限速。
lifecycle_manager	生命周期	统一配置、激活和关闭 Nav2 lifecycle 节点。

第十四章 Nav2 应该怎样接入 ROS2 与 Unitree SDK

这一章讲的是推荐架构：Nav2 仍然只面对 ROS2 标准接口，Unitree SDK 由独立 bridge 封装。这样做的好处是职责清楚、调试容易、后续替换控制器或机器人平台也更干净。

14.1 总体架构

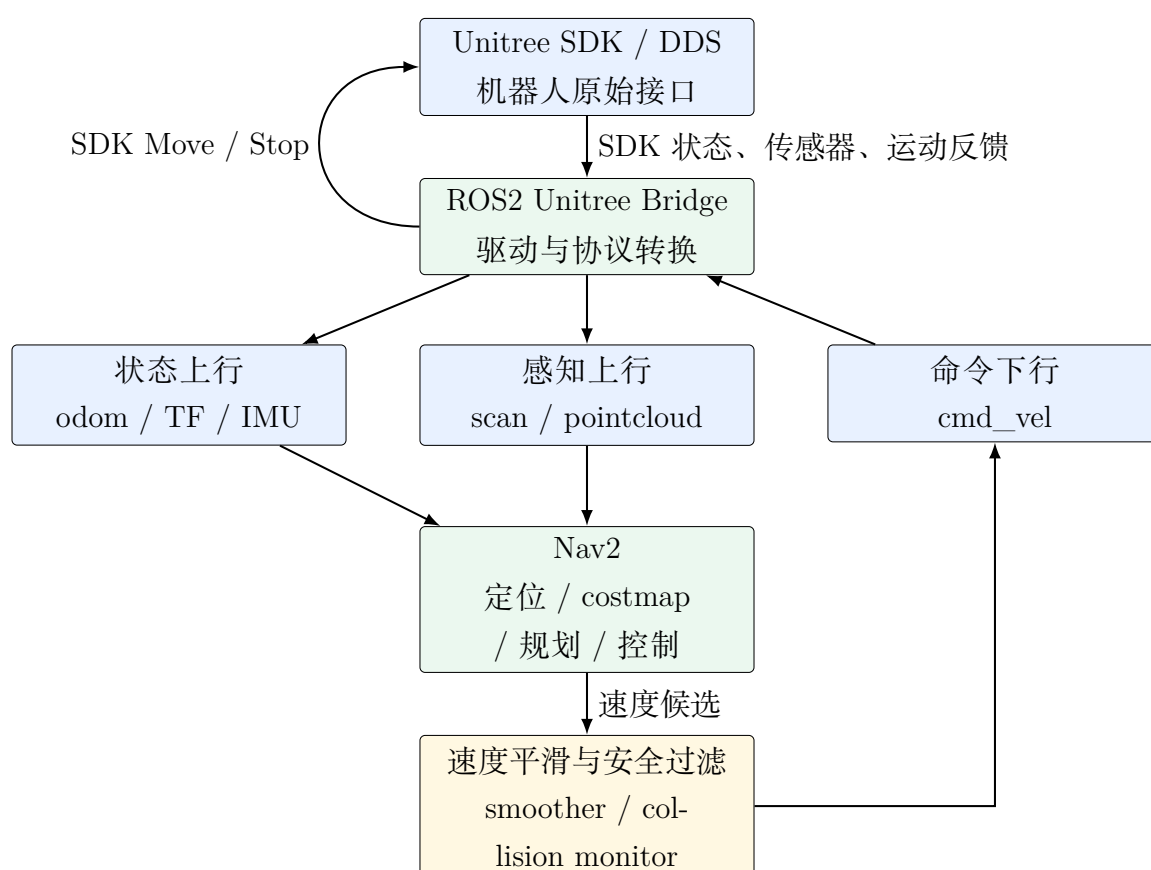


图 14.1: Nav2 与 Unitree SDK 的推荐接入方式：中间用 ROS2 bridge 隔离厂商接口。

14.2 Unitree Bridge 的上行职责

Bridge 从 Unitree SDK/DDS 读取原始状态，然后发布 Nav2 能理解的 ROS2 数据。最少应包含：

ROS2 输出	类型	作用
/odom	nav_msgs/Odometry	给 controller 和定位模块提供连续里程计。需要位置、姿态、线速度、角速度。
/tf	tf2_msgs/TFMessage	发布 odom -> base_link，必要时发布传感器外参。
/imu	sensor_msgs/Imu	给状态估计或算法使用，也可用于姿态和角速度参考。
/scan	sensor_msgs/LaserScan	2D 避障输入，适合 obstacle layer。
/pointcloud	sensor_msgs/PointCloud2	3D 避障输入，适合 voxel layer 或点云调试。

上行数据要特别注意四件事：

1. 时间戳必须来自同一时间基准，不能严重滞后。
2. frame 名称必须和 Nav2 参数一致。
3. 点云或雷达必须能通过 TF 变换到 costmap frame。
4. 里程计要短时间连续，不能频繁跳变。

14.3 Unitree Bridge 的下行职责

Nav2 输出的是 ROS2 标准速度命令，通常为：

```
geometry_msgs/msg/Twist
linear.x
linear.y
angular.z
```

Bridge 订阅 /cmd_vel 后，需要把它转换为 Unitree SDK 的运动接口调用。对四足机器人，可能存在两类执行方式：

- 普通运动接口：把速度直接转为厂商 SDK 的移动命令。

- 厂商避障运动接口：把速度交给 SDK 内部避障模块，让 SDK 在近距离做最后安全处理。

无论使用哪一种，Bridge 都应该实现：

- 速度限幅，避免 Nav2 参数错误导致机器人动作过激；
- command timeout，如果一段时间没有收到 `/cmd_vel`，立即发停止；
- emergency stop 或 enable 开关；
- 坐标约定转换，例如 Nav2 的 `base_link` 前左上约定与 SDK 约定是否一致；
- 日志与诊断，能看见当前收到的速度和实际发送给 SDK 的速度。

14.4 Nav2 与 Unitree SDK 的完整数据流

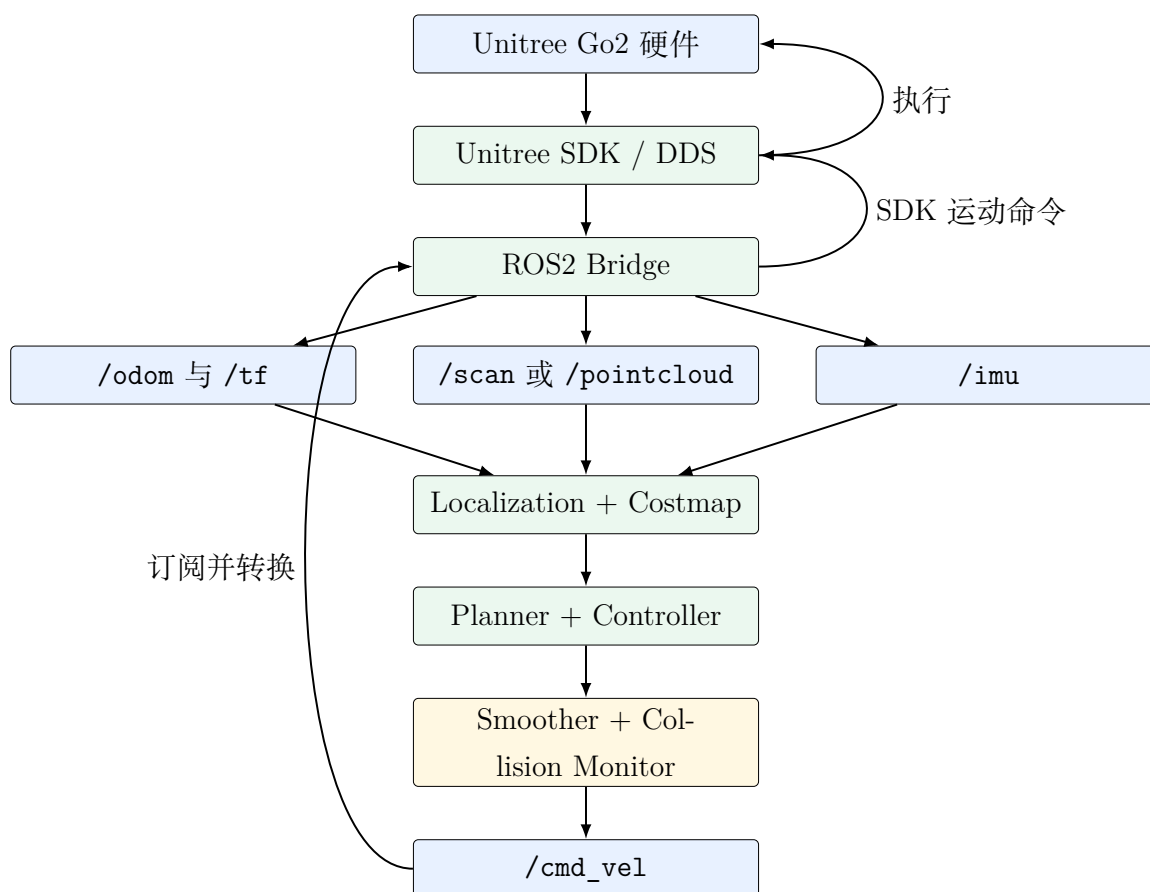


图 14.2: 从 Unitree 硬件反馈到 Nav2 决策，再回到底盘执行的闭环数据流。

第十五章 接入检查清单

15.1 ROS2 数据是否合格

接入 Nav2 之前，先确认下面数据已经稳定：

```
ros2 topic echo /odom
ros2 topic echo /tf
ros2 topic hz /odom
ros2 topic hz /scan
ros2 topic hz /pointcloud
ros2 run tf2_ros tf2_echo odom base_link
```

检查重点：

- /odom 是否连续；
- /tf 是否能查到 odom -> base_link；
- 传感器 frame 是否能变换到 base_link 或 costmap frame；
- 时间戳是否接近当前时间；
- 机器人静止时速度是否接近 0；
- 机器人前进时 linear.x 符号是否正确；
- 机器人左转时 angular.z 符号是否正确。

15.2 Nav2 参数是否合格

最小参数检查包括：

- global_frame 是否是 map 或 odom；
- robot_base_frame 是否与 TF 中的机器人本体 frame 一致；
- odom_topic 是否指向实际发布的 /odom；
- local/global costmap 的传感器 topic 是否存在；

- footprint 或 robot radius 是否符合机器人实际尺寸；
- controller 最大速度是否低于机器人安全速度；
- transform tolerance 是否能覆盖传感器和 TF 的延迟。

15.3 Unitree Bridge 是否合格

Unitree SDK 接入层应具备以下能力：

能力	检查方式
发布 odom	机器人移动时 /odom 位置和速度变化合理。
发布 TF	odom -> base_link 可查询，且不跳变。
发布传感器	RViz 能看到 scan 或 pointcloud，且位置和机器人一致。
订阅速度	手动发布小速度时，机器人按预期方向移动。
安全停止	停止发布速度后，机器人能在 timeout 内停下。
限幅	输入过大速度时，bridge 发给 SDK 的速度被限制在安全范围。
诊断日志	能看见 SDK 连接状态、当前模式、最后速度命令时间。

第十六章 从总体上记住 Nav2

最后用三句话总结：

1. Nav2 是 ROS2 上的导航决策系统，不是机器人厂商 SDK 的替代品。
2. Nav2 通过标准 ROS2 接口工作：TF、odom、sensor、map、Action、Service、cmd_vel。
3. 接入 Unitree Go2 时，正确边界是“Unitree SDK 由 bridge 封装，Nav2 只看 ROS2 标准数据，只输出标准速度命令”。

只要这个边界清楚，后面无论使用官方 waypoint follower、route server、DWB、RPP、MPPI，还是把 Unitree 内部避障作为最后执行层，都可以在同一套 ROS2 数据流上组合，而不会把导航逻辑和厂商 SDK 细节混在一起。